WaveVis inverse-sheet architecture for Moana-ocean breaking-wave linkage arrays

Alan N. Pham

AO Labs, WaveVis simulator, inverse deformation sheet record

WaveVis is an inverse simulator for a mostly flat rectangular linkage sheet with one localized plunging-wave deformation rising out of the sheet. The primary visual target is the supplied breaking-wave reference family: Moana-like water stills plus the June 23, 2026 line drawing with a broad outer arch, inward spiral curl, downturned hook tip, smooth inner throat, and long low return. The intended artifact is not an extruded ribbon, half-pipe, swept strip, roof, bridge, or hollow shell. It is a full array of linkage cells over a fixed flat perimeter: the center region ramps upward, forms a rounded crest, projects a lip forward and downward, curls into the hooked tip, and fades smoothly back into the surrounding sheet in all directions. The reference geometry is an acceptance target, not a success claim. A rendered state is still open if the human-facing render reads as a tunnel, wall-covered cavity, angular blob, missing curl, missing hook, or broken linkage even when code checks pass. A verification URL is also part of the artifact: if the page ignores URL slider parameters, the screenshot is not evidence for the state Alan requested. This paper replaces the earlier narrow constraint-proof PDF as the visible system record. It defines the reference geometry, coordinate systems, curated breaking-wave target, visible slider semantics, topology invariants, rendering views, failure modes, verification gates, deployment state, and handoff procedure required before WaveVis work can be called correct.

Fresh-chat use. This PDF is intended to be the first artifact a new WaveVis chat reads. It is not only a public paper. It is the continuation manual for the current simulator: what shape is being made, what failures were rejected, which files own the implementation, how each slider is allowed to behave, what tests must pass, what public route is verified, what remains unresolved, and how to continue without re-learning the same mistakes from chat history.

Contents

1 Introduction

WaveVis exists to explore whether a cellular Sarrus-style linkage array can approximate visually meaningful three-dimensional sheet deformations while preserving mechanical readability. The simulator therefore has two simultaneous jobs. First, it must generate a target shape that resembles the supplied Moana ocean references rather than a generic height field. Second, it must keep the linkage graph legible: cells, nodes, connector pairs, and edges must remain visible and connected rather than being hidden under cosmetic surfaces or erased when the shape becomes difficult.

The current design problem is stricter than drawing a pretty profile. A side silhouette can look acceptable while the actual three-dimensional sheet has a wall across the underside, a cavity where the lip should be open, an extruded barrel, or zig-zagging linkage legs that no longer read as a mechanism. Conversely, a mesh can satisfy a local numerical check while failing the human outcome: a full rectangular sheet with a localized plunging wave. WaveVis must therefore treat the visible artifact, the mechanism topology, the parameter semantics, and the regression checks as one architecture.

The central correction captured here is that a breaking wave is not made by sweeping one two-dimensional profile sideways. A swept profile creates a tunnel, canopy, or half-pipe. The required object is a localized deformation field on a flat sheet:

$$\Phi : (x, y) \mapsto \big(x + \Delta_x(x, y),\ y + \Delta_y(x, y),\ \Delta_z(x, y)\big),$$

where the displacement is concentrated near the middle span, gradually fades toward the lateral sides, and is exactly zero on the perimeter. The curl must be strongest near the centerline and weaker toward the sides. This spatial falloff is part of the target shape, not a rendering trick.

1.1 Reference geometry

The design reference is the Moana ocean plus the June 23, 2026 supplied line drawing: a smooth, localized volume of water that rises out of a larger flat body, grows into a broad rounded crest, curls into a smaller inward spiral, and finishes with a small downturned hook while leaving an open underside and a long low return. These reference frames are not decorative. They are the visual contract for the simulator's target geometry. A successful WaveVis state should read as a simplified linkage approximation of this water form: the outer radius is large and smooth, the inner curl radius is smaller, the lip projects forward before curling down, and the side field fades away instead of forming a constant barrel. This is deliberately stricter than matching a side-profile curve: the full linkage array in side and isometric views must preserve the open curl without side walls, erratic zig-zags, disconnected cells, or hidden filler geometry.

Current verification boundary. This document records the target, architecture, and required gates. It does not by itself prove that the current generator has matched the supplied references. The rendered full-array artifact remains the source of truth. If the page, screenshots, or live route do not visibly match the Moana-like localized plunging wave while preserving linkage cells and flat perimeter, the correct state is “open visual mismatch,” not “reference matched.”

Figure 1. Primary supplied reference geometry. The target form is a localized plunging water sheet with a rounded crest, open underside, and forward/downward lip. The WaveVis sheet must approximate this geometry while preserving flat perimeter boundary nodes and visible linkage cells.

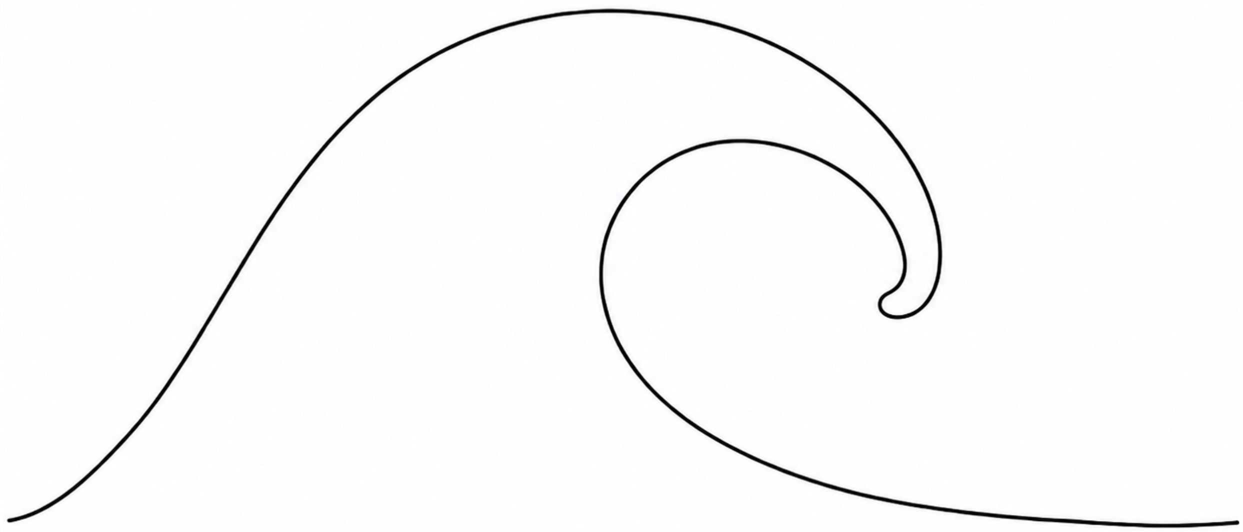

Figure 2. June 23 supplied side-silhouette reference. This image sharpens the visible target: broad outer arch, smaller inward curl, downturned hooked tip, smooth curved inner throat, and long low return. It is the current side-shape target, while the three-dimensional WaveVis render must still preserve the full linkage sheet rather than becoming a two-dimensional trace.

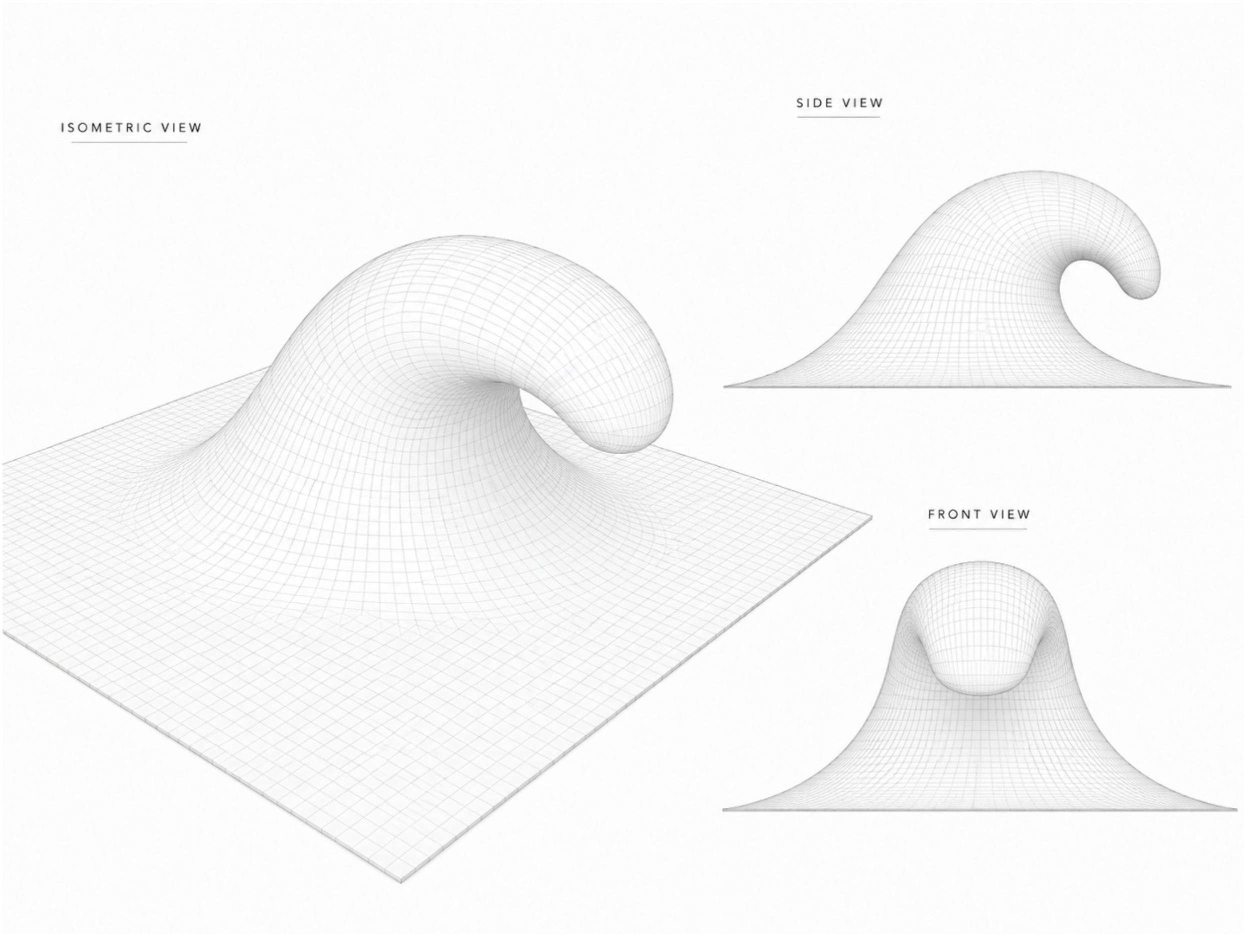

Figure 3. June 24 supplied smooth gridded reference overview. Alan re-supplied this image and said "remember." It is now a preserved project reference for the desired visual grammar: a continuous square sheet surface, rounded rising face, tube-like crest, downturned lip, open clean throat, and centered front view. A jagged linkage trace, side-wall tunnel, or metric-only improvement does not satisfy this reference.

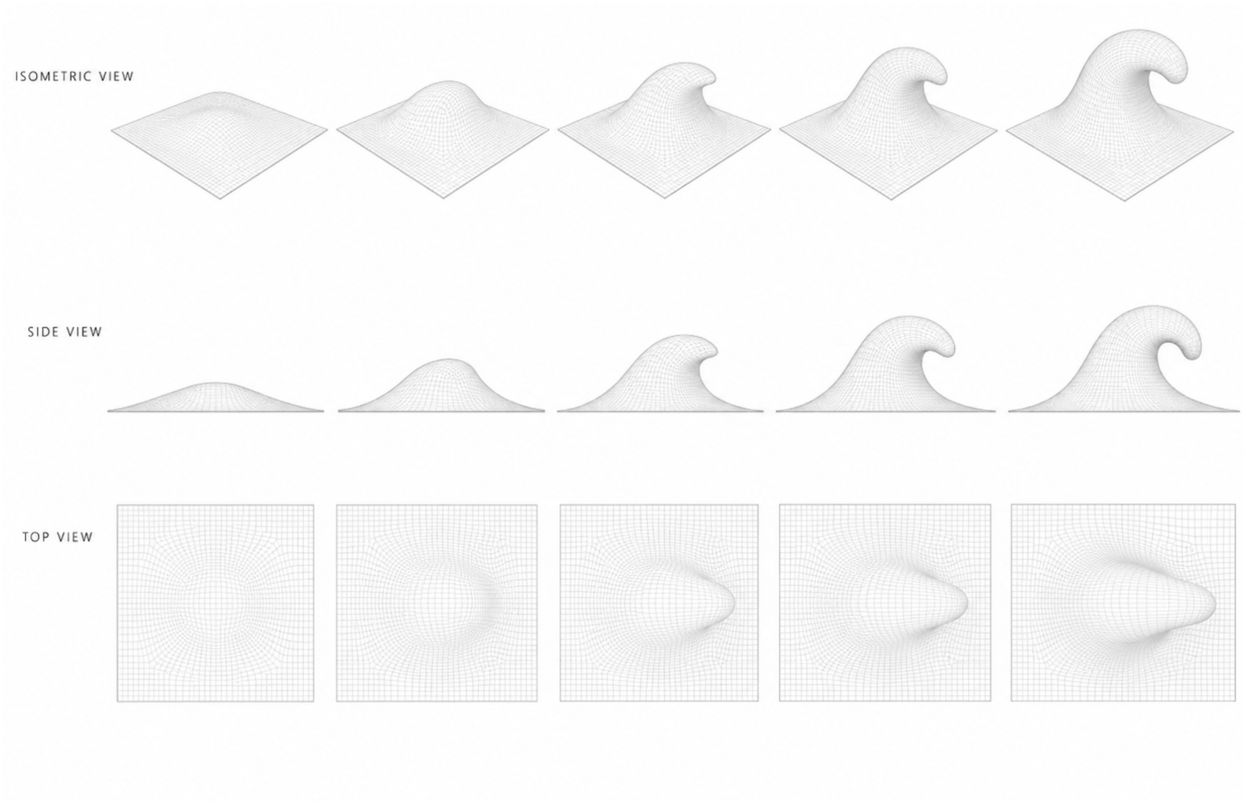

Figure 4. June 24 supplied smooth gridded reference sequence. The sequence records the intended progression across isometric, side, and top views: the deformation grows from a flat square sheet into a localized rounded wave with a curled lip while the top view remains a rounded localized body, not a triangular fan or swept strip.

2 Start here for a new WaveVis chat

If this PDF is the only WaveVis context available, start with this section and then inspect the live artifact. The current project state is:

- **Verified public route:** `https://aolabs.io/wavevis/`. This route serves the fallback bundle that Alan actually uses.
- **Preferred but blocked route:** `https://wavevis.aolabs.io/`. At the time of this record, HTTPS validation can fail because the custom-domain certificate is not valid for that hostname. Do not treat the custom subdomain as the verified route until a fresh certificate check passes.
- **Visible PDF action:** the WaveVis header links to the paper file under `/proofs/`. File: `wavevis-system-architecture.pdf`. The old connector proof remains a source artifact, not the primary user-facing paper action.
- **Current source root:** workspace path `_apps/wavevis.aolabs.io/`.

- **Public fallback mirror:** workspace path `_apps/aolabs-site/wavevis/`. A WaveVis deploy to its own `gh-pages` branch does not automatically update the `aolabs.io/wavevis/` fallback unless the built files are mirrored there and the hub site is pushed.
- **Current architectural direction:** curated custom Moana-ocean profile localized into the inverse-sheet field. Manual xz and xy profile editors are not the active path.
- **Current reference-match boundary:** the June 28, 2026 candidate536 readable shared-joint/top-footprint checkpoint targets the supplied smooth gridded breaking-wave reference in addition to the June 23 line drawing and older water stills. The default product view shows the readable surface plus the connected X-cell mechanism: surface is on, connector/X lines are on, and node/cell markers are off unless explicitly requested. The URL gates remain addressable with `surface=1`, `connectors=1`, and `cells=1`. Side is still a full three-dimensional side camera, not a cross-section. Candidate536 keeps the candidate528 recovered candidate330-style geometry floor, restored isometric camera, bounded 3D-only lower-lip tuck, `terminalTipTaper=0.1`, continuous top X-frame layer, candidate426 connected straight X-cell mechanism, candidate464 explicit shared connector joints, candidate492 instanced shared-joint rods, and the default/proof split introduced by the readable shared-X work. Candidate536 preserves the candidate529 Side/3D shared connector pin and real center-to-connector rod readability while reducing the normal Top surface-mode X-bar, joint, pivot, and arm overlay so the rounded plan footprint no longer becomes a dark terminal stripe. The physical instanced pin layer remains at every shared connector, and shared-joint marker sizes stay stronger than center pivots in Side and 3D so the between-X joints read as connector joints rather than implicit line intersections. Each shared connector still draws the two real center-to-connector arms that meet there; no fake bridge, filler layer, or surrogate contact path is introduced. Each cell has two straight diagonal bars, `sw-ne` and `se-nw`; each bar is bisected by the cell center; the two bars may have different total lengths and a free included angle; and neighboring cells share visible connector joints. The checkpoint is a guarded mechanism/readability improvement, not a completed reference match: Side and X-only views preserve the mechanism, Top is less visually overdrawn in default surface mode and no longer forms the heavy terminal stripe, Front remains a separate spanwise check, the full-array X mechanism remains visible, and 3D still reads blunter and more capped than the June 24 tube reference. The current X mechanism reports 4928 center pivots, 9240 rendered straight X segments, 19092 rendered shared-joint arms, 9546 shared connector joints, zero opposite-pair length spread, zero opposite collinearity error, zero center residual, zero

connector endpoint gap, and exactly two endpoint uses for every shared connector.

2.1 What the next agent must not repeat

The repeated failures that this document is meant to prevent are:

1. Do not make a swept tunnel, ribbon, canopy, roof, half-pipe, bridge, or extruded strip.
2. Do not form a bowl, dimple, cavity, pucker, or side wall that covers the underside of the curl.
3. Do not replace Side with a profile-only debug view. Side is a side camera view of the full three-dimensional sheet.
4. Do not hide, ghost, clip, or remove linkage cells because they look bad. Fix the geometry.
5. Do not remove the full array, straight X cells, shared connector joints, or pairwise linkage truth while chasing a prettier silhouette.
6. Do not wire the lip directly to the ground or draw filler geometry across the open underside.
7. Do not make `position` or `steer` shape controls. They are rigid post-shape transforms.
8. Do not call the work done from code intent, automated checks, a nicer profile line, or a local-only screenshot.
9. Do not let the top view collapse into a big triangle, wedge, or V fan while the side view looks acceptable.

2.2 Current live-state evidence

Figures 5–9 record candidate536 from the current browser-rendered source tree. The default presentation is a readable gridded surface plus the straight connected X-cell mechanism inspired by the June 24 reference: Side uses the source reference profile with the accepted raw curl profile, a light side wire grid, a very faint inner-lip silhouette, visible X diagonals, center pivots, stronger physical shared connector pins, and an instanced shared-joint rod layer, so the throat remains open without a fake bridge or false dark cavity outline; 3D uses the same source-profile readable surface with candidate528's recovered smoother geometry, lighter isometric skin, pale wire grid, bounded 3D face-pinch transform, reduced rounded-cap override, visible X cells, shared connector pins, and real center-to-connector rods. Front keeps the front-only readable projection with compressed depth, bounded terminal lip contribution, depth-tested pale grid lines, rounded terminal width, and the connected X mechanism visible through the center. Top keeps one square sheet panel and one continuous X-frame layer, but its default surface-mode shared joints, pivots, and arms remain lighter than the X-only proof so the visible terminal band does not become a black stripe. The mechanism is not deleted: `connectors=1` exposes the straight X linkage, `surface=0&connectors=1` exposes the X-only mechanism view, and `node scripts/check-inverse-sheet.cjs` verifies 4928 center pivots, 9240 rendered straight X segments, 19092 rendered shared-joint arms, 9546 shared connector joints, zero connector endpoint gap, and exactly two endpoint uses per shared connector. This paper therefore treats the current render as a clearer connected-X array with explicit between-cell joints, real center-to-joint rods, a less false-cavity side throat, and a less overdrawn Top default view while preserving the open exact-reference-match boundary.

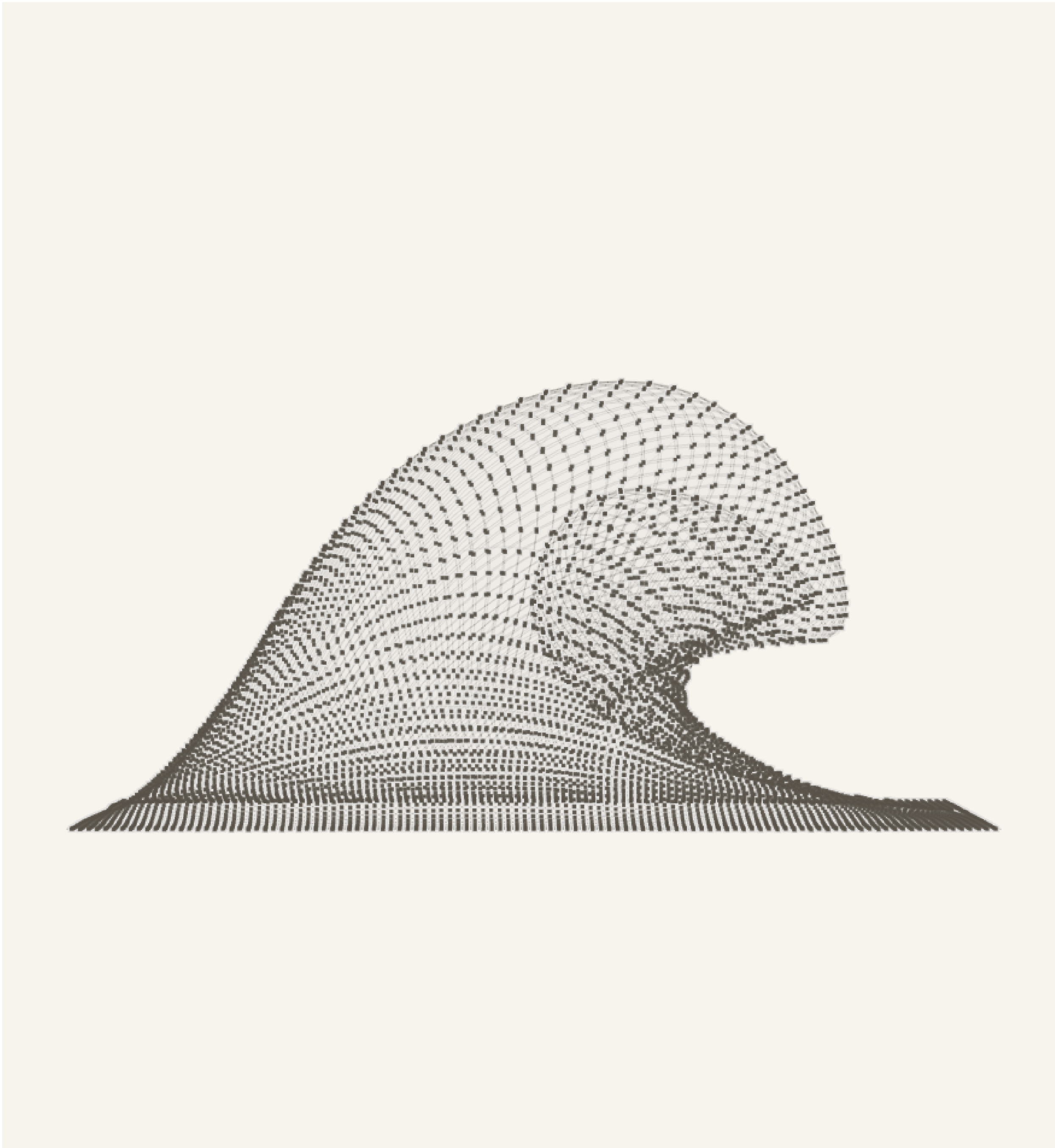

Figure 5. Candidate536 browser-rendered Side view for the default inverse-sheet state. The view is a full three-dimensional side camera with the readable surface, light side wire grid, straight X-cell diagonals, center pivots, physical shared connector pins, and instanced shared-joint rods visible together. It preserves the open exact-match boundary: the underside remains readable and the between-cell joints stay explicit, but the reference-wave match is still not closed.

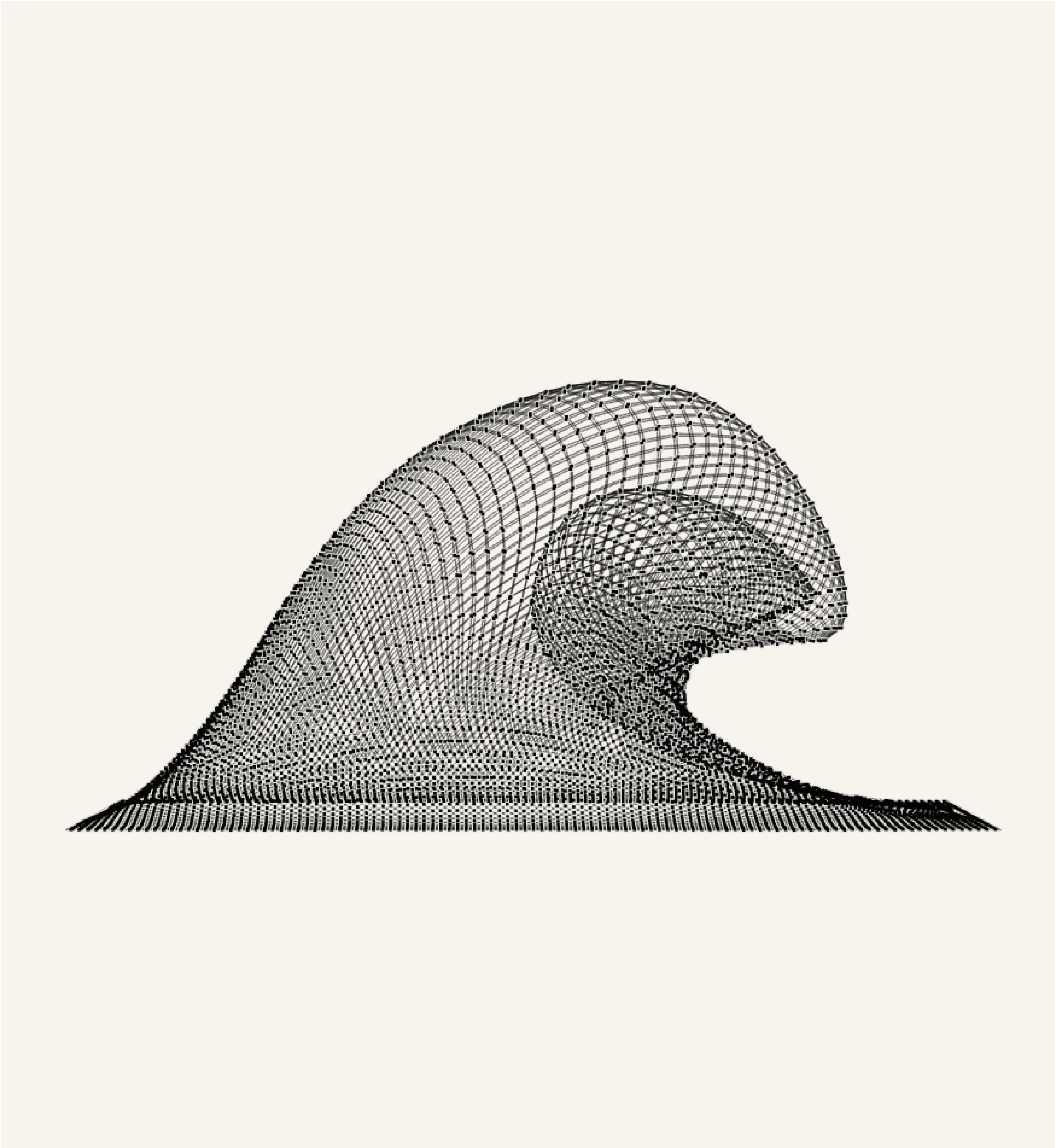

Figure 6. Candidate536 Side view with `surface=0&connectors=1`. This X-only evidence removes the surface skin and shows the actual mechanism layer: each cell is a straight two-bar X, each diagonal is bisected at the center pivot, and neighboring cells meet at visible shared connector pins through real center-to-connector rods.

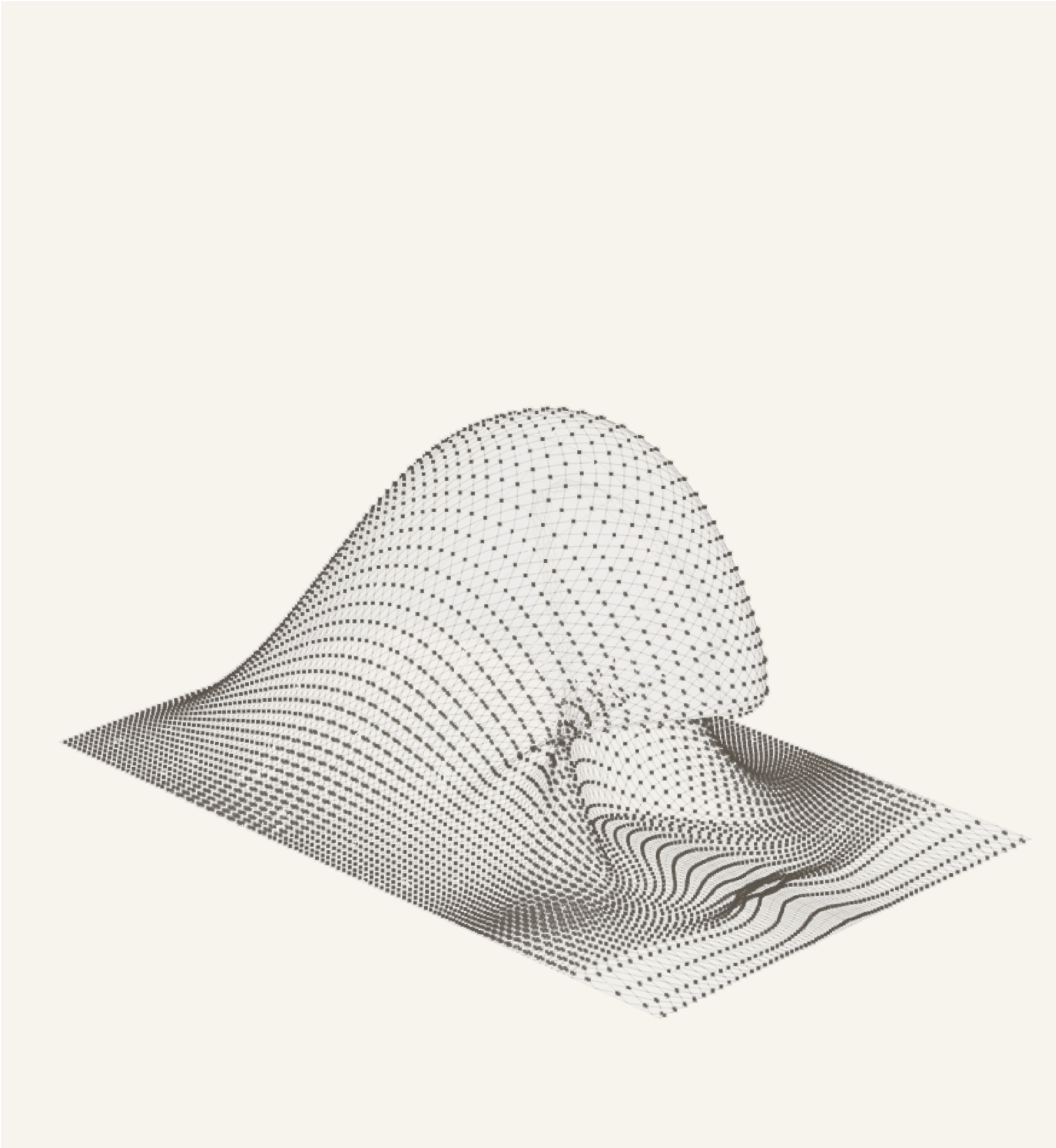

Figure 7. Candidate536 browser-rendered 3D/isometric view for the same default state. The default presentation retains the candidate528 recovered candidate330-style geometry floor, restored isometric camera, source-profile readable surface skin, lighter isometric surface, pale wire grid, bounded 3D face-pinch transform, connected straight X cells, physical shared connector pins, and a visible instanced shared-joint rod pass. Surface-on 3D preserves depth testing for hidden X/joint marks against the readable surface, reducing the false see-through throat wall without adding a fake bridge. This figure remains an improved open visual-match boundary rather than an exact reference match because the lip is still blunter and more capped than the June 24 tube reference.

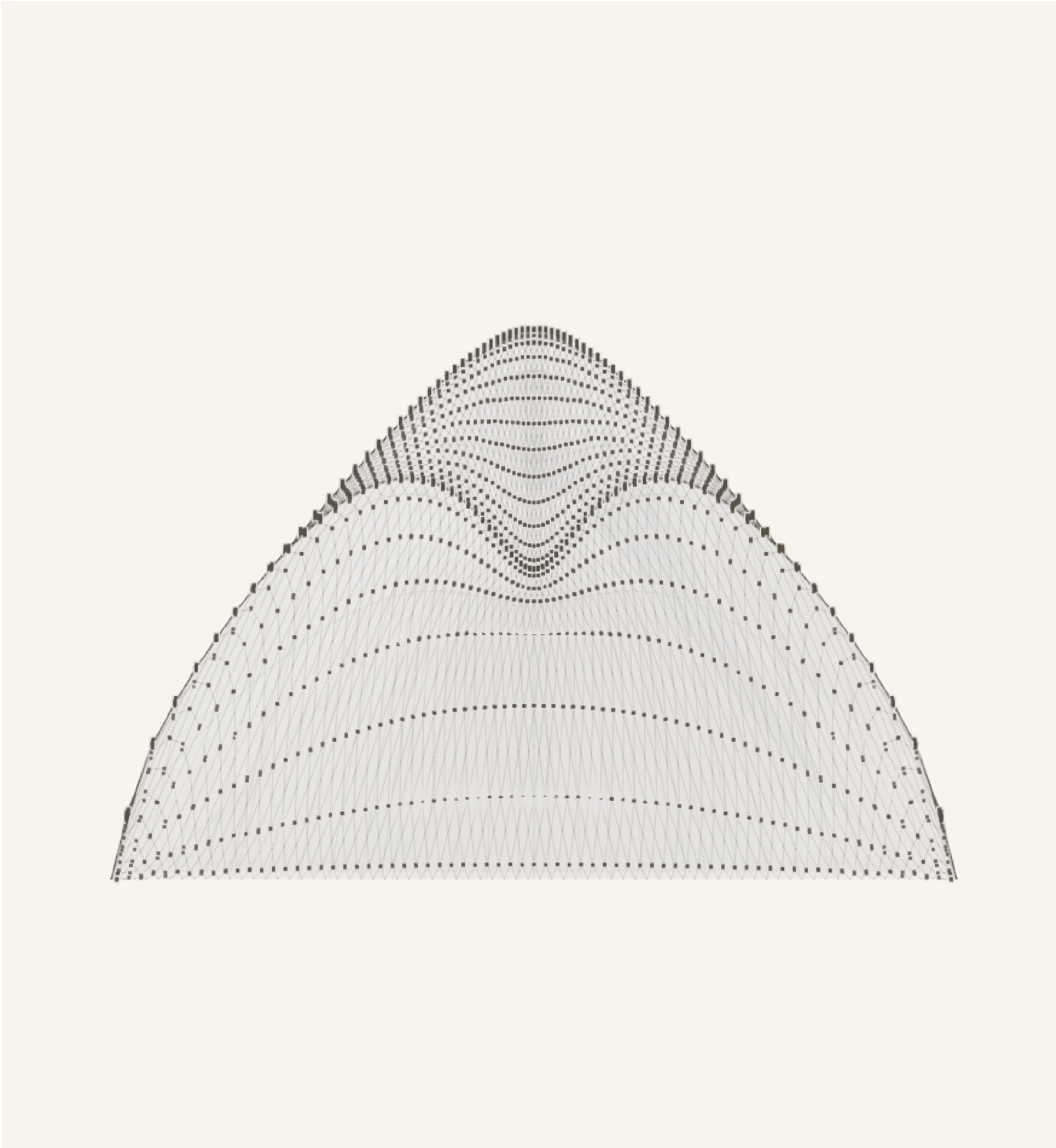

Figure 8. Candidate536 browser-rendered Front view for the same default state. This separate product camera compresses front depth, bounds the terminal lip contribution, depth-tests the pale grid against the lip, keeps rounded terminal width, and shows the center X-cell linkage through the projected cap rather than hiding mechanism state behind a pasted cap layer. The June 28 projection checkpoint remains open against the June 24 front reference.

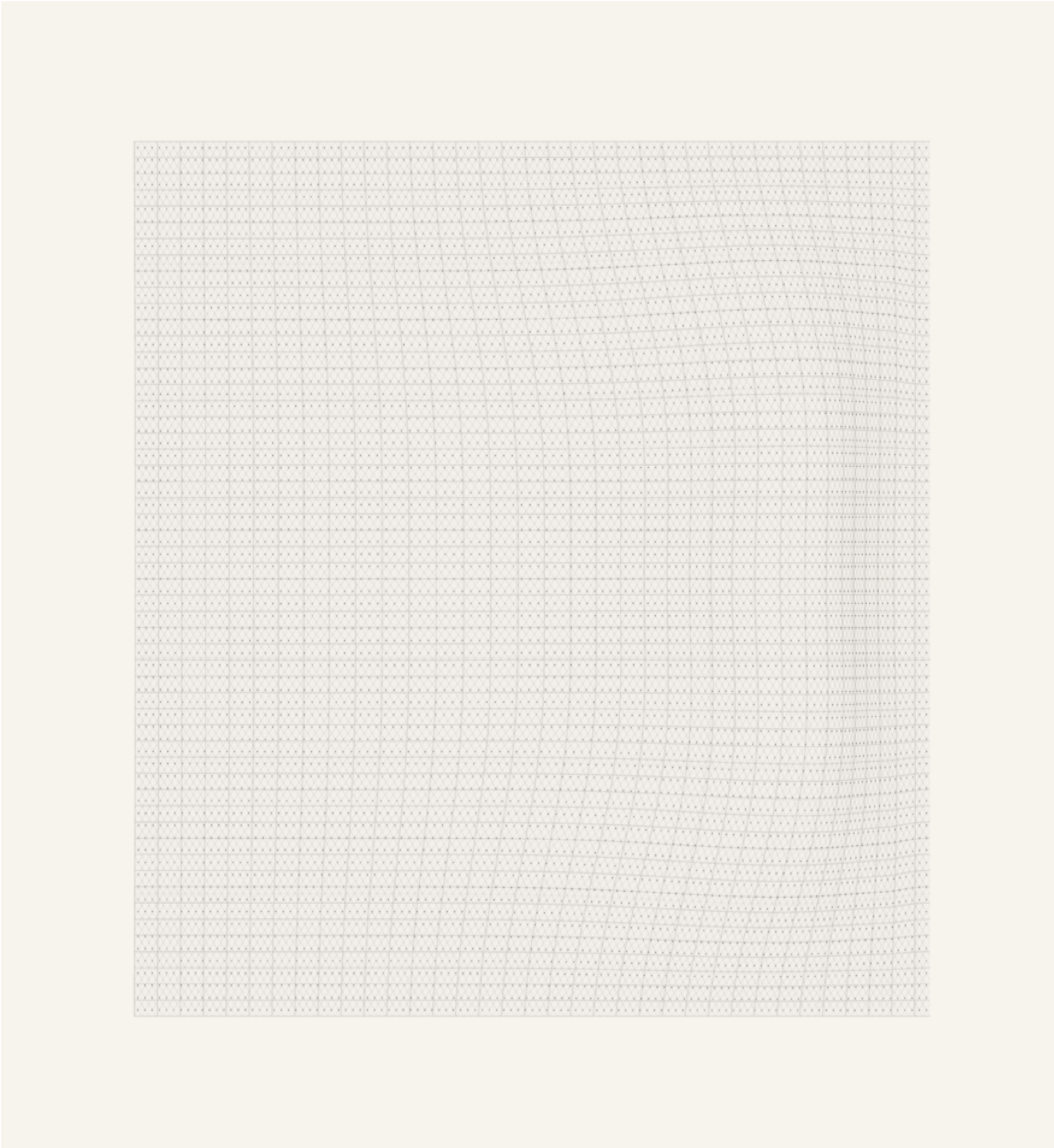

Figure 9. Candidate536 browser-rendered Top view for the same default state. The surface-plus-X view uses a Top-only shoulder-plus-terminal footprint projection with a continuous X-frame layer, visible shared connector pins, and real center-to-shared-joint rods. Candidate536 keeps the default Top surface-mode joints, pivots, and arms lighter than the X-only proof so the terminal linkage does not read as a heavy black stripe; strict `surface=0&connectors=1` top inspection still exposes the constrained X-only mechanism concentration rather than hiding it.

187 2.3 File map

File or route	Ownership
`src/latticeGeometry.ts`	Source of truth for the inverse sheet model, defaults, sanitization, target displacement field, morph behavior, metrics, and geometry sanity checks.
`src/LatticeViewer3D.tsx`	Source of truth for Canvas rendering, side/isometric/top full-sheet camera behavior, surface visibility, camera presets, visual layers, and the rendered straight X-cell, center-pivot, and shared-joint layers.
`src/xCellMechanism.ts`	Source of truth for the connected X-cell display mechanism: two straight diagonal bars per cell, equal opposite halves through each center pivot, and shared connector joints between neighboring cells.
`src/TargetShapeControls.tsx`	Source of truth for the visible inverse-sheet controls: rows, columns, views, scale, morph, position, steer, display toggles, reset, export, and import.
`src/InverseSheetTab.tsx`	Source of truth for URL-state loading, tab order, view requests, import/export plumbing, and the Inverse Sheet first-tab behavior.
`scripts/check-inverse-sheet.cjs`	Main inverse-sheet regression checker. It compiles the TypeScript geometry and checks parameter semantics, lip curl, flatContribution, low-lip stability, lateral smoothness, no bowl pockets, startup URL coverage, and the connected straight X-cell render contract.
`scripts/check-geometry-invariants.cjs`	Mechanism-level invariant checker for rigid-cell/linkage behavior.
`proofs/wavevis-system-architecture.tex`	Source for this PDF. Update it when the public simulator contract, current state, or known failure modes change.
`proofs/figures/`	Moana reference images, mechanism proof figures, and current source-rendered state figures used by this paper.
`dist/`	Built WaveVis static app after `npm run build`. Do not edit by hand.
`aolabs-site/wavevis/`	Public fallback mirror for `https://aolabs.io/wavevis/`. Copy the built `dist` contents here when the public fallback must update.

Table 1. Current source map. A new WaveVis chat should use these files rather than reconstructing behavior from old prompts.

2.4 Minimal continuation workflow

A fresh agent continuing WaveVis should do the following in order:

1. Read this PDF and `src/latticeGeometry.ts`, then inspect `src/LatticeViewer3D.tsx` and `src/TargetShapeControls.tsx`.
2. Open the verified public route `https://aolabs.io/wavevis/` and the current local route if a dev server is running.
3. Reproduce Alan's active preset from the URL before changing source. Do not judge a different default state.
4. Run `npm run check:geometry`. If it fails, fix source before visual tuning.
5. Render and inspect `Side`, `3D`, and `Top`. Side and 3D must both be full-sheet views with the curl visible.
6. Make the smallest source change that improves the human-facing Moana-wave outcome without regressing the mechanism invariants.
7. Re-run `npm run check:geometry` and `npm run build`.
8. Verify live/public output after deploy or fallback mirror update. A local screenshot is not the public artifact.
9. Update this PDF if the architecture, current state, source map, controls, or known failure classes changed.
10. Post a Progress event with complaint, issue, Codex change, observed source change, provenance, and commit ids.

3 Current implementation state

3.1 Defaults and target

The current inverse-sheet default is intentionally taller and sharper than earlier flat presets:

$$
\begin{aligned}
\texttt{rows} &= 44, & \texttt{columns} &= 112, & \texttt{morph} &= 1, \\
\texttt{height} &= 15.25, & \texttt{horizontalOffset} &= 22.5, & \texttt{overhangWidth} &= 32, \\
\texttt{overhangAngleDeg} &= 120, & \texttt{overhangPosition} &= -0.15, & \texttt{profileScale} &= 0.55, \\
\texttt{smoothing} &= 1, & \texttt{lipSharpness} &= 0.88, & \texttt{wallSmoothness} &= 0.28, \\
\texttt{flatContribution} &= 0.35.
\end{aligned}
$$

The app displays only the narrow active controls by default. Legacy inputs such as `height`, `overhang`, `lipDip`, `width`, `lipSharpness`, `groundTransition`, `wallSmoothness`, and `flatContribution` can still be parsed from URLs and JSON imports because older test links and screenshots use them. The visible startup path favors `scale`, `morph`, `position`, and `steer` to keep the main wave profile stable.

Columns are sanitized to a smooth-wave minimum. A URL may say `columns` = 24, but the current generator promotes the active sheet to the minimum column count needed for a smoother wave. This is intentional: too few columns produced angular side silhouettes, dimples, and visibly under-sampled curl segments.

3.2 Target-field construction

The target is constructed in a canonical wave frame. The canonical frame removes `position` and `steer`; the shape is computed there; then steer and position are applied as rigid transforms afterward. This prevents the old bug where moving the wave also changed its profile.

The source sequence in `buildNodes` is:

1. build rest grid p^0_{ij};
2. compute core target positions from `targetFromDeformationField`;
3. pin perimeter nodes to rest;
4. apply span continuity smoothing for generated mode;
5. apply flat-contribution diffusion as a support apron, not as a flattening blend;

6. interpolate from rest to target with `morphGrowthAmount`;
7. build metrics, edges, quads, and dihedral pairs from the resulting grid.

The target function samples u along the wave field and v across span:

$$u = \frac{x - x_{\min}}{L}, \qquad v = \frac{y + S/2}{S}.$$

The displacement is zero outside the support field and on the perimeter. The support field expands with `smoothing` and `flatContribution`, but the core wave must remain the same shape.

3.3 Rigid controls

`position` and `steer` are post-shape placement controls:

$$\begin{aligned} \Delta x_{\text{position}} &= 0.045\, L \operatorname{clamp}(\texttt{position}, -1, 1), \\ \psi_{\text{steer}} &= 45^\circ \operatorname{clamp}(\texttt{steer}, -1, 1). \end{aligned}$$

They must not modify the curated wave profile, width, curl, strain, or topology. Validation aligns geometries back to the canonical frame and checks that residual shape differences are near zero for position and bounded for steer.

3.4 Morph control

`morph` is not a shape recipe. It interpolates from rest to the completed target:

$$p_{ij}(m) = (1 - g(m))p^0_{ij} + g(m)p^*_{ij}.$$

The current growth function combines a sine ease with a faster visible growth curve so the wave grows naturally without a dead early range. Morph must preserve topology, connector closure, and the visible wave identity throughout the range. If $\texttt{morph} = 0.5$ produces zig-zags, overlaps, missing cells, or a side-wall-covered curl, that is a geometry failure.

3.5 Flat contribution and ground transition

`flatContribution` does not mean flatten the wave. It means share deformation with the surrounding sheet through a support apron. In accepted behavior:

- the main wave height and overhang reach stay within a small residual;
- the centerline profile remains essentially unchanged;
- surrounding flat areas participate more gradually;
- strain concentration is reduced or spread rather than hidden.

The old behavior `target` = lerp(`deformed`, `rest`, `flatContribution`) is explicitly rejected because it turns `flatContribution` into a flattening slider.

`smoothing`, shown historically as ground transition, broadens the support field and root ramp. It should recruit surrounding flat sheet into the slope while keeping the perimeter pinned and preserving the terminal lip.

3.6 Lip dip and terminal curl

`lipDip` maps to `overhangAngleDeg`. Neutral is 90°. High curl is 120°. The accepted behavior is not endpoint lowering. The wave should rise, crest, reach forward, curl downward, then return smoothly to the flat sheet without closing into a bowl. The current validation checks:

- the selected lip nose is a mid-height visible point below the last peak, not the near-floor return;
- the terminal slope is negative;
- the tip is forward of the crest and tucked under the shoulder by a bounded amount;
- the return advances toward the flat front instead of folding backward;
- `openThroatVisible`, `noBackfoldCavity`, and `noVisibleDimplePocket` remain true.

These checks are guards, not substitutes for the human-facing screenshot. A visible dimple, pocket, side wall, or missing curl still means open work.

3.7 Rendering contract

`LatticeViewer3D.tsx` renders the readable surface and a connected X-cell mechanism from the actual lattice graph. In the current contract:

- `view=side` sets a side camera over the full three-dimensional linkage array;
- `view=front` sets a front camera over the same full three-dimensional sheet so the spanwise lip cap can be inspected;
- `view=isometric` shows the full rectangular sheet in perspective while still making the curl and lip readable;
- the default product view uses a smooth gridded surface skin plus straight X-cell connector lines, while node markers remain available as a separate cell toggle;
- Side, Front, and 3D use view-specific mesh outlines instead of the previous SVG overlay or red terminal trace;
- each rendered X cell is made from two straight diagonal segments, `sw-ne` and `se-nw`;
- each diagonal is bisected by the cell center pivot, but the two diagonals do not need the same length or a right angle;
- neighboring X cells meet at shared connector joints, and every shared connector must have exactly two endpoint uses;
- cells and connectors are display toggles, not repair paths, and their render scope remains covered by the geometry check.

If the renderer needs to hide rows to make the shape readable, the generator is still wrong.

(a) Open underside and suspended lip.

(b) Forward/downward curl at the water edge.

(c) Localized mound emerging from flat water.

(d) Rounded column without a hard perimeter wall.

Figure 10. Additional supplied reference frames. Together they define the acceptance target: water-like surface continuity, strong center curl, smooth lateral fade, no side wall covering the underside, and no swept tunnel extrusion.

4 Target outcome

4.1 Human-facing shape

The accepted target is a “flat sheet plus localized plunging wave”:

- the outer perimeter of the rectangular sheet remains flat and fixed;
- the wave emerges through a broad, smooth ramp rather than a vertical wall;
- the crest is rounded, with no top dent;
- the lip projects forward and curls downward;
- the open underside is visible from side view, not covered by a side wall;
- the shape fades back to flat in every direction;
- the array remains a full linkage field with nodes and legs, not a hidden or clipped shell.

Figure 11 gives the practical acceptance boundary. The shape must read as a local wave in a sheet, not as an extrusion. A single side profile can guide the curl, but the generated three-dimensional field must vary across span.

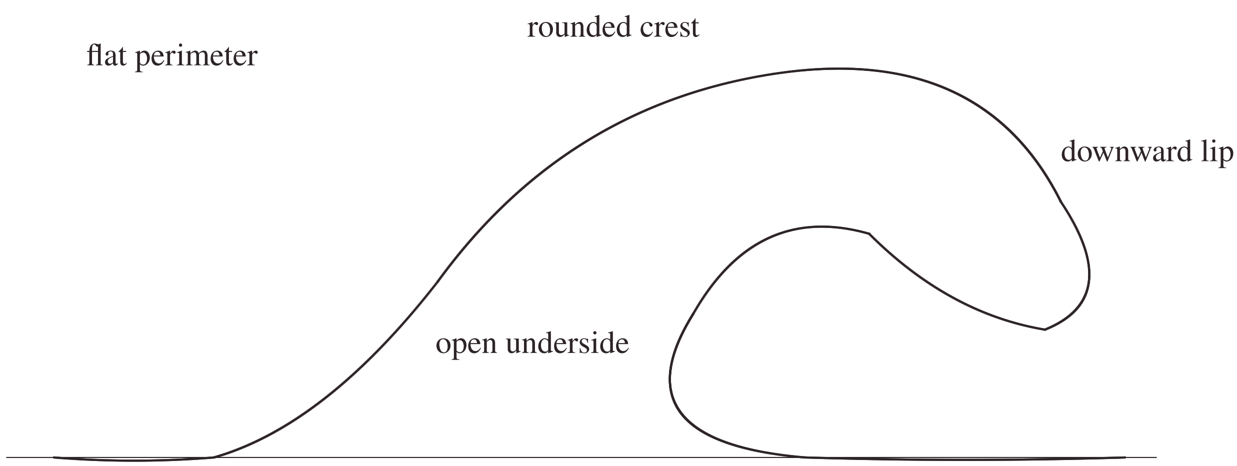

Figure 11. Desired side silhouette as an outcome object. The drawing is not a literal mesh recipe. It is the visible side-view contract: smooth back ramp, larger outer radius, smaller inner radius, curled lip, and curved interior throat. The three-dimensional solver must then localize this silhouette in the middle of a full sheet rather than sweep it as a constant tunnel.

4.2 Hard constraints

The WaveVis generator and renderer must preserve the following hard constraints:

1. **Flat perimeter.** Boundary nodes must remain on the rest sheet. No side, front, or rear perimeter should be pulled into the wave.
2. **Full array visibility.** The rendered artifact must show the actual linkage grid. Hiding or ghosting broken cells does not count as a fix.
3. **Connector closure.** A cell connector is an actual shared graph location. Lines should meet at nodes; apparent detached legs are a failure.
4. **No erratic legs.** Long arms, spikes, zig-zags, random backtracking, and visible overlaps indicate a broken geometry or render path.
5. **Smooth strain sharing.** Adjacent rows and columns should change gradually enough that the mesh reads as a continuous mechanism field.

6. **Localized wave.** The curl must be strongest near the center and fade laterally. A constant side-to-side tunnel is not the target.

7. **Full-view side semantics.** Side view shows the full three-dimensional render from the side. It must not be replaced by a profile-only debug view.

8. **No filler geometry.** Cosmetic walls, caps, planks, bridges, hidden couplers, or masks must not be used to conceal a bad mechanism.

5 Coordinate model

The simulator starts from a rectangular rest lattice. A node with row i and column j has rest position

$$p_{ij}^0 = (x_j, y_i, 0).$$

The target is a displacement field

$$p_{ij}^* = p_{ij}^0 + m\, D(x_j, y_i; \theta),$$

where $m \in [0, 1]$ is the morph amount and θ represents all shape parameters except rigid placement. The morph parameter only interpolates between rest and the finalized target. It must not change the target shape definition, remesh the lattice, or introduce a different topology.

The longitudinal coordinate $u \in [0, 1]$ measures progress through the wave region from rear/root to front/lip. The span coordinate $v \in [-1, 1]$ measures lateral distance from the centerline. A good WaveVis target uses both:

$$D(x, y) = A(u, v)\, P(u) + B(u, v)\, Q(u),$$

where $P(u)$ is the side-profile contribution, $Q(u)$ can include forward curl or secondary displacement terms, and A, B are smooth two-dimensional envelopes that go to zero at the sheet perimeter.

The key architectural rule is that $A(u, v)$ is not constant in v. A constant span envelope turns any attractive profile into a tunnel. A localized plunging wave requires $A(u, 0)$ large near the centerline and $A(u, \pm 1) \approx 0$ near the sides, with a monotone or smoothly unimodal falloff rather than a hard wall.

6 Generation pipeline

The target-building pipeline is:

1. sanitize rows, columns, visible sliders, hidden legacy shape parameters, display flags, and URL parameters;
2. build the full rectangular node grid;
3. compute the canonical Moana-ocean wave displacement from the curated target;
4. apply the span envelope that localizes the wave in y;
5. preserve the boundary by forcing perimeter displacement to zero;
6. apply morph interpolation from rest to target;
7. apply rigid steer and position transforms after the shape is finalized;
8. build full rectangular edge and quad topology;
9. compute strain, bend, shear, dihedral, and display metrics;
10. render the full array, side view, 3D view, and top view without substituting fake surfaces.

The distinction between the curated target and rigid placement parameters is essential. The visible app no longer exposes manual xz or xy profile editors. The source target owns the Moana-ocean profile. The user-facing sliders are intentionally narrow: scale uniformly resizes that target, morph blends from rest to the completed target, steer rotates the finished target around the anchor, and position translates the finished target horizontally. Neither steer nor position may feed back into profile curvature, curl, width, strain generation, or cell topology.

Stage	Contract	Failure caught
Target profile	Defines the desired side silhouette.	Blob, neck/head, hill without lip, top dent.
Span envelope	Localizes the wave in the sheet.	Swept barrel, side wall, U-shaped tube blocker.
Boundary clamp	Keeps the perimeter flat.	Pulled sheet edges, non-flat perimeter.
Morph	Interpolates rest to target only.	Half-morph mangling, topology change while sliding.
Topology	Keeps full node/edge/quad graph.	Missing grid, hidden cells, detached legs.
Renderer	Shows the actual mechanism.	Fake shell, hidden wall, side view as profile-only debug output.

Table 2. Pipeline stages and the failure modes each stage must prevent.

7 Curated Moana Target

The current visible app does not ask the user to draw a profile. That path created too much ambiguity and repeatedly made the simulator look like a swept tunnel or hand-edited cavity rather than a single coherent wave. The user-facing target is now driven by a curated custom Moana-ocean lip/body profile: a localized ocean deformation with a broad rear ramp, rounded crest, forward-and-down lip, open underside, smooth lateral fade, and flat rectangular perimeter.

The curated target is a reference field, not a set of cell locations. The final cell grid is determined by row and column counts, then sampled against that target. A clean state must keep the full linkage array visible while making the centerline and the lateral silhouette read as one plunging wave. The target must never stitch the lip directly to the ground or draw a hard polygon boundary around a former editor curve, because that creates the wall, bowl, and cavity failures this record explicitly rejects. The curl is also resolution-sensitive: under-resolved column counts are promoted to the smooth-wave minimum because honoring a compact column request recreated the visible dimple and made the side silhouette fail.

8 Slider semantics

Control	Intended effect	Must not do
`morph`	Blend rest sheet into the completed target.	Remesh, flip cells, create zig-zags, change profile identity.
`scale`	Uniformly enlarge or reduce the curated Moana-ocean target.	Change steer, position, topology, or the target's wave identity.
`steer`	Rigid yaw rotation of the finished wave.	Bend, stretch, or reshape the wave.
`position`	Rigid horizontal translation of the finished wave.	Change silhouette, strain field, or curl.
display toggles	Surface, flat grid, cells, and connectors are display layers only.	Hide broken geometry or substitute a fake repair path.

Table 3. Visible control contract after removing manual profile editors and hidden shape sliders. The curated Moana-ocean target is the default shape; visible controls must preserve that shape unless their narrow role explicitly changes scale, morph, steer, position, or display layers.

9 Breaking-wave geometry

The desired default profile is closer to a localized plunging wave than to a cone, cylinder, or arch. The useful approximation is a tapered, asymmetric surface whose highest crest sits behind the curl, whose outer radius is larger than the inner radius, and whose underside remains open. Along the centerline, a breaking wave profile can be represented as three joined curve families:

$$P(u) = \begin{cases} P_{\text{ramp}}(u), & 0 \le u < u_c, \\ P_{\text{crest}}(u), & u_c \le u < u_l, \\ P_{\text{lip}}(u), & u_l \le u \le 1. \end{cases}$$

The ramp rises from the sheet. The crest rounds over. The lip turns forward and downward. The derivative at the terminal lip should point downward when `lipDip` $> 90^\circ$:

$$\left.\frac{dz}{dx}\right|_{\text{tip}} < 0.$$

The final point is allowed to be lower than the farthest horizontal reach. In a real breaking wave, the maximum reach can occur at the shoulder while the lip tip turns down from that shoulder. The downturn must remain open and forward-readable from the side; it must not fold into a closed pucker, dimple, bowl, or side-wall pocket. Treating the farthest-right point as the tip is the mistake that produces a straight falling chute, while overcorrecting into an under-tucked return is the mistake that produces the rejected cavity. The underside return must also advance back into the flat sheet after the lowest lip sample; a backward floor point is disallowed because it reads as a small lower-lip dimple even when it does not trip a larger bowl metric. The current repair keeps the stronger visible dip by lowering the pendant return and rounding the cap, but bounds the lower branch so the full side render remains a localized plunging sheet rather than a swept tube.

The three-dimensional field must multiply this centerline motion by a span falloff:

$$E(v;u) = \exp\left[-\left(\frac{|v|}{w(u)}\right)^{\alpha}\right],$$

with $w(u)$ varying through the wave. The curl should have a smaller active width than the broad back ramp, so the lip is localized and the open underside remains visible from the side.

10 Mechanism topology

WaveVis uses a rectangular graph of nodes, horizontal profile edges, vertical span edges, and quads as the source lattice. The visible mechanism layer now derives a connected X cell at each lattice node. Each cell has a center pivot and two straight diagonal bars. The `sw-ne` bar is one bisected line and the `se-nw` bar is the other bisected line. The two bars may differ in total length, and the included angle is free. Neighboring cells share the diagonal connector locations, so the array reads as a field of X cells connected through visible joints rather than as isolated crossed marks. The graph is not allowed to become a set of isolated rows.

The mechanism evidence from the earlier proof remains relevant. The all-four-equal X-cell branch was rejected because the finite connector assignment does not close cleanly under the current overhang contract. The accepted branch is pairwise: one straight bar provides one equal collinear opposite pair, the crossing bar provides the other equal collinear opposite pair, and the angle and length relation between the two bars are allowed to vary. This pairwise rule preserves a readable connected X linkage without forcing impossible equal-arm closure (*1*).

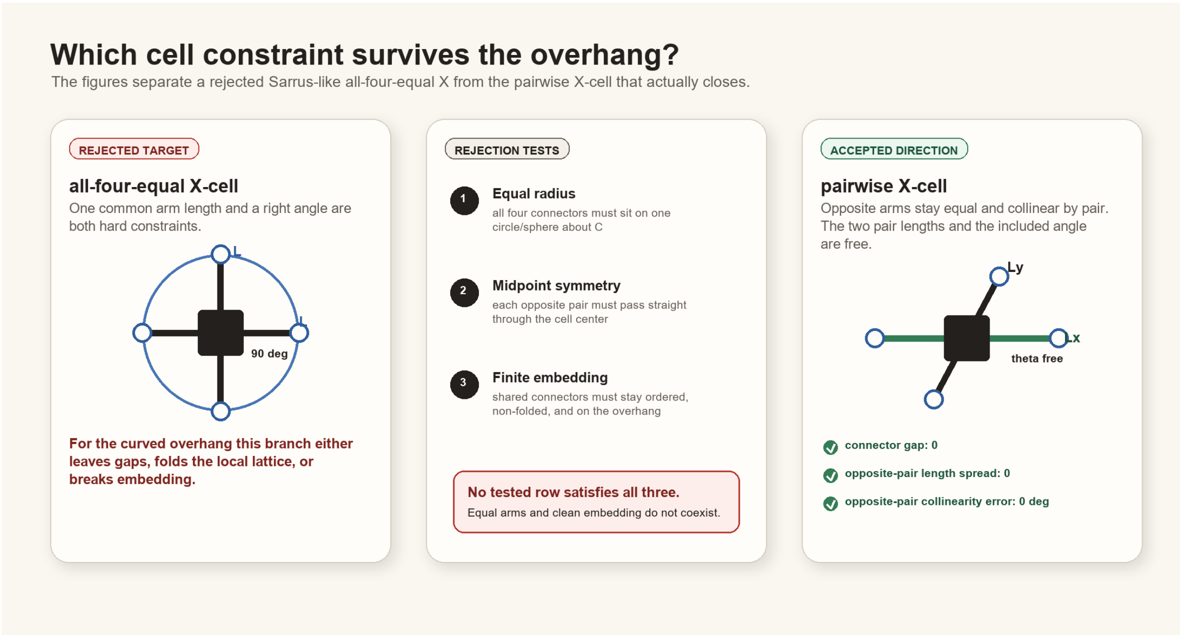

Figure 12. Mechanism contract inherited from the constraint proof and updated in the current renderer. WaveVis should preserve exact shared contact, straight X bars, and pairwise equal, collinear opposite halves rather than hiding infeasible all-four-equal geometry behind filler surfaces.

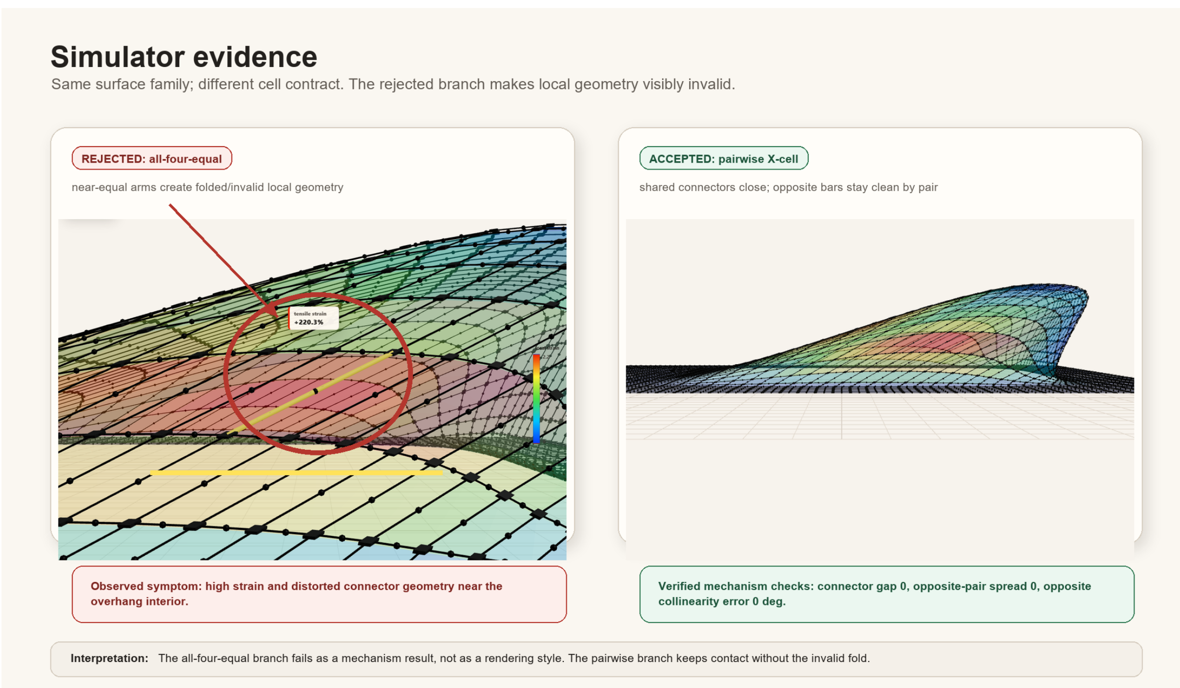

Figure 13. Simulator evidence for the linkage direction. The current architecture must keep this mechanism truth visible while the inverse target shape is improved.

11 Rendering modes

403

WaveVis has three product views:

- **3D/isometric view** shows the full rectangular array in perspective.
- **Side view** is a camera view of the full three-dimensional render from the side and must show the curl.
- **Top view** shows the full square plan sheet with localized interior deformation and catches terminal caps, triangular fans, wedges, or cropped-patch regressions.

The view contract matters because a side view can hide the curl if lateral geometry forms a wall over the open underside. In that case the correct fix is to reshape the three-dimensional surface so the lateral envelope follows the wave contour and fades out, not to switch the side view into a profile-only debug output or hide the wall.

12 Verification gates

The generator should be checked with both numeric and visual gates. Numeric checks are not sufficient, but they catch repeated hard failures before visual review.

Gate	Measurement	Acceptance intent
Perimeter flatness	$\max_{\partial\Omega} \|p^* - p^0\|$.	Boundary displacement is zero or visually negligible.
Topology completeness	Expected node, edge, and quad counts.	Full rectangular sheet graph remains visible.
Connector closure	Edge endpoints meet their node coordinates.	No detached legs or fake bridges.
Lip curl	Crest height minus lip-nose height; terminal slope; `terminalFaceHasRun`; `openThroatVisible`.	The lip curls down and leaves an open throat rather than ending as a hill or near-vertical chute.
Span localization	Row-wise height profile across y.	Center strong, sides faded; no swept tunnel.
Morph continuity	Residual between adjacent morph samples.	No sudden jumps or topology flips.
Slider isolation	Aligned geometries across position/steer/width changes.	Rigid controls stay rigid; width stays span-only.
Visual side check	Browser side view screenshot or direct render inspection.	Curl visible in the full side view.
Defect repair loop	Confirmed visible defects re-enter source repair.	Acknowledging dimples, pockets, walls, or missing curl is not a stopping state.
`noVisibleDimplePocket`	Open downturn, return advancement, and curl clearance in the centerline.	The tip may curl downward, but it must not form a tight pucker, pocket, dimple, or enclosed bowl.
Startup URL parser	Covers all public and legacy slider aliases.	A shared verification URL cannot silently load a different height, overhang, lip dip, width, or smoothing state.

Table 4. Verification gates. A build passing TypeScript is not enough; WaveVis must pass the rendered outcome Alan actually uses.

12.1 Current regression command set

The current source-level gate is:

```
npm run check:geometry
```

which expands to:

```
node scripts/check-geometry-invariants.cjs
node scripts/check-inverse-sheet.cjs
```

The inverse-sheet checker compiles the TypeScript geometry into a temporary CommonJS module and then evaluates the default state, low/high lip dip, morph, position, steer, flat contribution, smoothing, width, lateral taper, bowl-pocket, side-render, and startup-URL cases. The geometry-invariant checker covers mechanism-level consistency.

The current acceptance evidence expected from `scripts/check-inverse-sheet.cjs` includes:

- startup display defaults to `showSurface=true`, `showEdges=true`, `showNodes=false`, and `showHeatmap=false`;
- rendered center pivot count equals the full rectangular grid count 4928;
- rendered straight X segment count equals the expected interior X segment count 9240;
- rendered shared connector joint count equals the expected diagonal connector count 9546;
- each shared connector has exactly two endpoint uses;
- maximum X connector endpoint gap is zero;
- maximum X opposite-pair length spread is zero;
- maximum X opposite collinearity error is zero;
- maximum X opposite-center residual is zero;
- every interior X cell exposes the two connected diagonal pairs required by the mechanism;
- connected X-cell centers stay close to the sampled wave surface rather than becoming a detached fake mechanism;
- no bowl-pocket count is reported for default, compact-user, and breaking-wide presets;
- side-overhang aspect stays in the downturned-lip band after the supplied-line update: startup default aspect between 1.35 and 2.65, compact user morph case between 1.45 and 2.85, and both with reach-to-height between 0.72 and 1.35;
- broad open-curl lip stats keep open-throat, no-dimple, no-backfold, and return-to-flat checks true;
- those lip stats allow a wide rounded cap and tucked nose instead of forcing the old vertical-chute proxy;
- slider robustness reports no bad cases.

These checks are designed around Alan's repeated failures. If a future change removes one of these cases because it is inconvenient, that is a regression unless the PDF and source explain an intentional replacement gate that still catches the same failure.

12.2 Rendered verification

Numeric checks must be followed by rendered verification. The minimum visual pass is:

1. render the exact user URL/preset, not a nearby default;
2. check `Side`: full 3D side camera, visible curl, no profile-only substitution, no wall covering the open underside;
3. check `3D`: full-sheet perspective view, visible curl/overhang, localized center deformation, no swept tunnel, no hidden rows;
4. check `Top`: the sheet should remain square while the interior footprint localizes toward the curl; it must not become a constant extruded strip, broad terminal cap, clipped patch, or large triangular fan;
5. inspect cells/connectors at the curl for zig-zags, random long arms, detached legs, overlaps, or missing four-leg interior connectivity;
6. check lower `lipDip` values as well as 120°, because old low-lip cases became unstable even when max lip looked acceptable;
7. check `morph`=0, 0.5, and 1 because half-morph repeatedly exposed mangled intermediate states.

Screenshots are evidence only when they show the actual rendered canvas and the URL or visible controls identify the tested state. A screenshot of a nice profile line is not evidence that the full linkage array is correct.

13 Build, deploy, and publication runbook

13.1 Local development

Use the WaveVis app root:

`_apps/wavevis.aolabs.io/`.

Typical local commands are:

- `npm run dev` to run the Vite development server;
- `npm run check:geometry` to run geometry and mechanism checks;
- `npm run build` to build the static app;
- `npm run deploy` to publish the app's own GitHub Pages branch.

Do not manually edit `dist`. Build it from source.

13.2 Public fallback deployment

The route Alan can reliably use is `https://aolabs.io/wavevis/`. That route is served from the AO Labs hub/fallback repository, not directly from the WaveVis app repository. Therefore a public WaveVis update requires two publication steps when the fallback route is the user-facing route:

1. build and publish the WaveVis app;
2. copy the built `dist` contents into `_apps/aolabs-site/wavevis/`;
3. commit and push the AO Labs site mirror;
4. verify that `https://aolabs.io/wavevis/` serves the new bundle hash and the rendered artifact.

This is not optional. A source commit or WaveVis `gh-pages` deploy can still leave Alan looking at a stale `aolabs.io/wavevis/` bundle.

13.3 Custom-domain boundary

`https://wavevis.aolabs.io/` is the preferred branded route, but a hostname/certificate failure means it is not the verified artifact. The current Progress source separates `wavevis_home` from `wavevis_custom_domain`. Keep them separate. Do not claim the custom subdomain is fixed until a fresh request verifies the certificate and body from that hostname.

13.4 Progress logging

After meaningful WaveVis work, write a Progress event before final response when practical. The event must include:

- `source_ids`: usually `wavevis_home` and `wavevis_custom_domain`;
- Alan's complaint/request;
- the issue being solved;
- the Codex change;
- the observed public source change;
- provenance: thread, commits, checks, screenshots, and deployment basis;
- uncertainty: exact match not proven, custom domain stale, or any unverified surface.

Progress does not automatically ingest every active Codex chat. If a future WaveVis chat uses this PDF and changes the app, it must still write the event itself.

14 Current verified and open state

This section is intentionally blunt so a future chat does not have to infer whether the system is finished.

Verified current state on June 28, 2026:

- The reliable public route is `https://aolabs.io/wavevis/`.
- The public WaveVis page includes a clickable AO Labs home mark, an app identity mark, the `wavevis.aolabs.io` title, and a `System Architecture PDF` action.
- The Inverse Sheet tab is the first/default option; Double-Layer Sarrus is secondary.

- The current PDF route is the WaveVis fallback paper route: `/wavevis/proofs/`, file `wavevis-system-architecture.pdf`.
- Side, Front, Top, and 3D are the product view modes. Side is the full three-dimensional sheet seen from the side, not a profile-only debug view.
- Current checks pass for the production build, mechanism geometry invariants, and the inverse-sheet regression suite.
- The June 28 candidate536 readable shared-joint/top-footprint checkpoint uses a default surface-plus-X presentation across 3D, Side, Front, and Top, hidden-by-default node markers, visible connector/X lines, preserved mechanism toggles, slider robustness, connected X-cell invariants, a June 24 reference-trace source profile, the restored isometric camera, retained 3D-only lower-lip tuck, a stronger center lip tuck, a very faint Side inner-lip silhouette, a light Side wire grid, a lower Front projection dome, a continuous top X overlay without the old terminal split, physical instanced shared connector pins between neighboring X cells, the candidate492 instanced shared-joint rod layer, candidate464 explicit shared joints, candidate477 surface-on 3D depth-tested hidden X/joint rendering, candidate528 recovered candidate330-style 3D geometry, and candidate536 preserves stronger Side/3D shared-joint/pivot/arm rendering than candidate528 while keeping the normal Top overlay lighter than the X-only proof view.
- The current side render uses a left-to-right side camera with a smooth rise, reference-trace forward/downturned inward curling lip, shared connector joints, overhang, and an X-only verification path through `surface=0&connectors=1`. Candidate536 keeps the side wire grid and side throat helper pale enough that they do not draw the separate dark false-cavity outline while leaving the actual connected X mechanism visible. The current 3D render keeps the sheet visible as a gridded surface with connected X cells and avoids the rejected hidden-line, bulb, hard-plateau, lit-oval, throat-shadow patch, opacity-only smoothing branch, overstrong diagonal-crease transform, front-biased flattened camera, open-C cavity, forward/down vertical-insert profile, busier profile throat-smoothing branch, opposite-camera branches, and the candidate404 side-projection skirt. It depth-tests hidden X/joint marks in surface-on 3D so backside linkage does not fake a dark throat wall. It remains visibly short of exact June 24 reference identity because the lip is cleaner but still blunter and more capped than the supplied tube reference. The current Front render is a spanwise check with depth-tested pale grid lines, rounded terminal width, controlled cap contours,

a front-only pale lit cap cue, a lower projection dome, and visible center X linkage, not a closed reference match. The top render keeps one square reference sheet and uses the rounded plan branch with one continuous X layer; candidate536 keeps the default surface-mode shared joints, pivots, and arms lighter than the X-only proof while still making the between-X connector points visible.

- Browser-rendered side, X-only side, front, isometric, and top figures were refreshed from candidate536. Current source metrics include 4928 center pivots, 9240 rendered straight X segments, 19092 rendered shared-joint arms, 9546 shared connector joints, zero X connector endpoint gap, exactly two endpoint uses for every shared connector, zero X opposite-pair length spread, zero X opposite collinearity error, zero X opposite center residual, max X center-surface residual 0.4596, and source guards requiring the visible shared-joint rod layer plus the bounded default Top surface-mode joint split.

Open state that must not be overstated:

- Exact matching to the supplied breaking-wave line drawing, June 24 gridded reference, or photorealistic Moana-water frame is not a proven completed result. The correct claim is that the simulator now moves closer to the requested full-mesh curl, tip, and overhang without regressing full-array linkage.
- `https://wavevis.aolabs.io/` is still treated as blocked/stale unless a fresh certificate and body check passes.
- This is a simulator and architecture record. It is not evidence that a physical lattice prototype has been fabricated or experimentally validated.
- If Alan reports a visible dimple, cavity, side wall, bowl, missing curl, zig-zag, disconnected cell, or profile/side confusion after this PDF ships, the live rendered artifact overrides this paper. Repair the source and update the validation gate.

15 Failure modes

The most important failure classes are now explicit:

1. **Swept barrel.** One side profile is dragged sideways at constant span. The result reads as a tunnel, roof, half-pipe, or bridge.

2. **Side-wall cover.** The lip may exist in a center profile, but full side view shows a wall covering the open underside.
3. **Straight chute.** The crest drops nearly vertically to a flat shelf instead of curling forward and under.
4. **Blob or lump.** Excess smoothing or broad support creates a mound with no wave identity.
5. **Top dent.** The outer crest has a kink or concave dent where a smooth radius is required.
6. **Dimple, cavity, or bowl.** The surface creates a pocket, pucker, enclosed hollow, or bowl-like depression rather than a plunging sheet.
7. **Stale URL state.** A browser URL contains height, overhang, width, lipDip, or smoothing values, but startup ignores them and renders a different state.
8. **Zig-zag linkage.** Neighboring legs alternate direction or length erratically.
9. **Hidden mechanism.** Broken topology is ghosted, clipped, or replaced by a cosmetic surface.
10. **Passive defect acknowledgement.** A visible problem is named in chat or recorded in this paper, but the generator and rendered artifact are not repaired and rechecked.

The solution direction is to simplify the target field while strengthening the gates: localized smooth wave first, full array second, mechanism-clean render third. Complexity should be added only if it improves one of those outcomes without breaking the others.

16 Materials and Methods

The implementation source is the WaveVis React and TypeScript app (*2*). The main geometry source is `src/latticeGeometry.ts`. The primary render source is `src/LatticeViewer3D.tsx`. Controls are declared in `src/TargetShapeControls.tsx`. Regression checks are run through `scripts/check-inverse-sheet.cjs`.

The public app serves a static build. The architecture PDF is placed in the public proofs directory as `wavevis-system-architecture.pdf` and linked from the WaveVis control headers. The old connector proof remains a source artifact for the mechanism decision, but the visible paper action now points to this architecture record because the current work is broader than connector assignment.

17 Discussion

The main design lesson is that WaveVis cannot be corrected by tuning only the side profile. The user-facing failure was three-dimensional: side walls, swept tubes, missing full-array linkages, broken side-view semantics, and zig-zag legs. The architecture therefore now uses a single curated target contract rather than a manual editor contract. The sheet generator is responsible for creating a localized Moana-like displacement field with lateral fade, boundary preservation, topology completeness, and a visible curled lip.

This distinction also clarifies future work. If the simulator produces a good center profile but a bad side view, the profile is only evidence that the curve exists somewhere. The real artifact still fails if the full three-dimensional render hides the curl or reads as an extrusion. Similarly, if the mesh is clean but the wave looks like a low mound, the mechanism is not enough. Both the shape and the linkage must be correct.

A newer lesson is that defect naming is not progress by itself. If a rendered state shows dimples, pockets, side walls, or a missing curl, the correct workflow is source repair, geometry-check update, rendered screenshot verification, and public-bundle verification. The current check names this directly as `noVisibleDimplePocket`: the lip must remain an open downturned branch with visible clearance and a return that advances into the flat sheet instead of folding into a pucker. The same source-of-truth rule applies to URL state. Several correction screenshots used URLs that carried explicit height, overhang, width, lipDip, and smoothing values; ignoring those parameters caused the page to render a different geometry than the requested test state. The startup parser is now part of validation through `startupUrlParamCoverage`. The architecture record is useful only if it changes the generator, state loader, and acceptance loop; it cannot substitute for a corrected human-facing WaveVis render.

18 Acknowledgements

This record was written to reduce repeated handoff burden during WaveVis development. The repeated complaints about side walls, missing cells, zig-zag legs, profile-view confusion, and swept-barrel geometry are treated as acceptance criteria rather than optional polish notes.

19 Funding

No external funding is claimed in this system architecture record.

20 Author contributions

Alan N. Pham defined the target artifact, accepted and rejected visual states, and the mechanism constraints. Codex generated this architecture record from the current WaveVis source and the active development contract.

21 Competing interests

The author declares no competing interests for this internal architecture record.

22 Data availability

The source data for this record is the WaveVis source tree, the Alan-supplied Moana ocean reference images copied into the proof figure set, the local proof diagrams, and the current simulator behavior. The public PDF is served with the WaveVis static app.

23 Code availability

The relevant implementation files are `src/latticeGeometry.ts`, `src/LatticeViewer3D.tsx`, `src/TargetShapeControls.tsx`, and `scripts/check-inverse-sheet.cjs` in the WaveVis app repository.

24 Additional information

This document supersedes the old header link to the narrow connector-contact constraint proof as the primary public WaveVis PDF. The connector proof remains an implementation reference for mechanism feasibility, but the current public artifact needs the broader system architecture and outcome-verification contract.

25 References

1. A. N. Pham, *WaveVis overhang connector assignment: evidence rejecting all-four-equal X-cells*, Internal WaveVis constraint proof and simulator evidence figures, 2026.

2. A. N. Pham, *WaveVis inverse-sheet simulator source tree*, Local source files: src/latticeGeometry.ts, src/LatticeViewer3D.tsx, src/TargetShapeControls.tsx, scripts/check-inverse-sheet.cjs, 2026.